

Chapter 9. Multimodal Large Language Models

```
!pip install -q transformers==4.46.3 accelerate sentencepiece
```

```
# It installs the Hugging Face transformers library version 4.46.3,
!pip install -q transformers==4.46.3
```

```
# ===== OpenCLIP =====
```

```
# imports the urlopen function from urllib.request module
# urlopen() = opens the given url and download its contents
# from urllib.request import urlopen

# PIL is a library used to work with images in Python.
# The Image class is used to open, create, edit, process, and save images in Python.
from PIL import Image

# Instead of using urllib to load the image from a URL (which gave an error),
# I uploaded the image to Colab and loaded it from the local path.
# Load an AI-generated image of a puppy playing in the snow
puppy_path = "/content/puppy.jpg"

# open the url form the given file path and converts each pixel into RGB colour format which is expected by model
image = Image.open(puppy_path).convert("RGB")

# create a caption to the image , later it will converted to embeddings and compares with the image embeddings
caption = "a puppy playing in the snow"
```

```
#from transformers library importing
# CLIPModel = The CLIP model is used to understand both images and text together.
# It converts both into embeddings (vectors) in the same vector space so they can be compared.
# CLIPTokenizerFast = tokenizer for the CLIP model , Converts text into token IDs
# CLIPProcessor = Preprocesses images and/or text into the format expected by the CLIP model (pixel values and input IDs).
from transformers import CLIPTokenizerFast, CLIPProcessor, CLIPModel

# This is the model we want to use
model_id = "openai/clip-vit-base-patch32"

# Load a tokenizer to preprocess the text
clip_tokenizer = CLIPTokenizerFast.from_pretrained(model_id)

# Load a processor to preprocess the images
clip_processor = CLIPProcessor.from_pretrained(model_id)

# Main model for generating text and image embeddings
model = CLIPModel.from_pretrained(model_id)
```

Loading weights: 100%

398/398 [00:00<00:00, 9344.62it/s]

```
# Tokenize the caption and retrun the result in pytorch tensor values
inputs = clip_tokenizer(caption, return_tensors="pt")
inputs
```

```
{'input_ids': tensor([[49406, 320, 6829, 1629, 530, 518, 2583, 49407]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1]])}
```

```
# Convert our input back to tokens
# Now we can understand what each ID represents.
clip_tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
```

```
['<|startoftext|>',
 'a</w>',
 'puppy</w>',
 'playing</w>',
 'in</w>',
 'the</w>',
 'snow</w>',
 '<|endoftext|>']
```

```
# Generate a 512-dimensional text embedding from the tokenized input using the CLIP text encoder.
# **inputs unpacks the input dictionary (input_ids and attention_mask) and passes them to the model.
text_embedding = model.get_text_features(**inputs)
print(text_embedding)

BaseModelOutputWithPooling(last_hidden_state=tensor([[[ 0.3393, 0.1165, 0.1020, ..., 0.2468, 0.5906, 0.1013],
[ 1.9753, -0.5844, 0.3685, ..., 1.1658, 0.8050, -0.9801],
[ 1.2633, -0.1076, 1.6844, ..., -0.6631, 0.4620, -1.4851],
...,
[ 0.8036, 0.0317, 1.7370, ..., -0.9917, -0.1352, 0.3876],
[ 0.9536, -0.0742, 1.7721, ..., 0.1970, 1.5288, 0.1174],
[ 0.4674, 0.6058, 2.1394, ..., -0.9849, 0.0902, 1.0475]]]],
grad_fn=<NativeLayerNormBackward0>), pooler_output=tensor([[-2.1203e-01, -1.6565e-01, -1.7938e-01, 1.1072e-02, -9.
-2.3305e-01, 2.4848e-01, -2.9610e-01, -3.0062e-01, -2.9995e-01,
-2.9974e-01, -1.1562e-01, 8.6049e-02, -1.1517e-01, 2.3986e-01,
-7.9785e-02, -1.0922e-01, 5.5739e-02, -2.2396e-02, 2.7013e-01,
-3.4473e-01, 5.0725e-02, 3.4548e-01, -1.0751e-01, 3.5118e-01,
9.8836e-02, 3.1391e-01, 4.2872e-01, 6.8715e-02, -2.2540e-01,
4.7834e-02, -1.3867e-01, 2.2445e-01, 2.9130e-02, -1.6404e-01,
-1.2395e-01, -2.5013e-01, 1.8177e-01, -2.5448e-01, 1.8794e-01,
-1.4910e-01, 2.9965e-01, -2.7238e-02, 2.7347e-01, 4.3485e-01,
2.5773e-01, -1.4844e-01, -5.4257e-01, 3.2448e-01, 1.7072e-01,
2.3203e-01, -4.1036e-02, -2.2473e-01, 2.0191e-01, 3.1726e-01,
-2.9506e-01, 5.2746e-02, 4.4709e-01, -3.5914e-01, -1.4772e-01,
1.3678e-01, 1.4624e-02, -1.8115e-02, 2.9516e-01, 4.8609e-02,
-2.7505e-01, -1.8307e-01, -3.3685e-01, -4.4555e-01, -1.1419e-01,
8.3711e-02, -5.3520e-02, 1.9751e-01, 2.3458e-02, -2.6442e-01,
2.2070e-01, 2.4041e-01, 2.5026e-01, 1.2394e-01, -7.1720e-02,
-9.5793e-03, 2.6620e-01, -2.7057e-01, 7.5659e-02, 3.7313e-01,
-1.3868e-02, 2.6065e-01, 2.1559e-01, -8.7918e-02, -9.6384e-02,
-1.0615e-01, -2.3696e-02, -4.7007e-01, 2.5713e-01, -7.0641e-02,
2.6900e-02, -4.4044e-01, 4.3292e-01, -3.1314e-01, -3.4131e-01,
5.6483e-01, 7.2882e-01, 2.8985e-01, 4.7358e-01, -2.2662e-01,
-2.8005e-01, -1.1723e-01, -6.0221e-02, -3.1428e-02, -5.1241e-02,
2.6184e-01, -4.0719e-01, -1.8572e-01, -2.0671e-02, -7.5803e-02,
2.1086e-01, -3.1142e-01, 1.1538e-01, -8.9314e-02, -1.7299e-01,
-8.5310e-02, -7.7920e-01, -2.4224e-01, -5.7324e-01, -1.8418e-01,
-4.0933e-01, 1.8472e-02, -2.9080e-01, -2.0909e-01, -9.1832e-02,
6.8372e-02, -2.7946e-01, -3.7376e-01, 3.1641e+00, -8.1386e-01,
-2.7317e-01, 9.4526e-03, -2.7301e-01, 3.0505e-02, -1.9140e-01,
-2.7708e-01, -4.4148e-02, -1.6650e-01, 2.4229e-01, -6.7603e-01,
-1.2839e-01, -7.1997e-02, -1.1923e-01, -1.2159e-01, 2.2136e-01,
-2.7137e-01, -2.0111e-01, -5.2321e-01, 5.9730e-01, 4.5413e-02,
-7.7909e-02, 1.3385e-01, -3.6563e-01, -7.2953e-02, 8.8675e-02,
3.2603e-01, 1.0657e-01, 1.8563e-01, -1.1004e-01, -2.9664e-01,
-1.7385e-01, 1.9004e-01, -3.4088e-01, 2.8821e-01, 9.0428e-02,
2.7621e-01, -7.1263e-02, 9.8975e-02, -1.5123e-01, 3.0950e-02,
5.1998e-02, -1.2670e-01, 1.6757e-01, 1.9013e-01, 1.2754e-01,
1.1103e-01, 3.0405e-03, -3.4231e-01, -3.2426e-01, 2.9565e-01,
-4.2420e-02, -2.0462e-01, 9.3709e-02, -2.8935e-01, 1.9211e-01,
-5.7343e-02, -3.1985e-01, -2.7905e-01, 5.1351e-01, -1.1474e-01,
2.6042e-01, -8.0023e-02, 3.8322e-01, -5.0478e-01, -6.1922e-02,
-5.8655e-02, 1.5313e-01, 1.1206e-01, 5.8275e-02, -5.5394e-02,
-4.4193e-01, 3.9421e-01, -3.3911e-01, -4.1290e-02, 5.2131e-02,
1.1952e-01, 3.3259e-01, 2.0935e-01, 1.3603e-01, -4.1564e-02,
1.8269e-01, 1.6856e-02, 2.8563e-01, 4.2763e-01, 1.2275e-01,
1.1804e-01, -3.0248e-01, 1.6450e-01, -3.9924e-01, 3.4350e-02,
-2.5335e-01, 7.9578e-02, -2.3579e-02, 1.2787e-01, 1.0046e-01,
3.4697e-01, 1.9920e-01, 4.7840e-01, 3.6180e-01, 4.0011e-02,
6.7191e-01, 3.5778e-01, -3.7259e-01, -1.3765e-01, -7.2665e-01,
-1.9359e-02, 2.9790e-01, 2.5572e-01, 2.5451e-01, -2.8003e-02,
-1.3693e-01, 3.6952e-01, 6.1645e-02, 1.0532e-01, 3.1322e-01,
4.3707e-01, 1.0700e-01, 2.6400e-01, 0.7110e-01, 0.5557e-02])
```

```
# Preprocess image
processed_image = clip_processor(

    # No text input, only the image will be processed.
    text=None,

    # Pass the input image to the processor.
    images=image,

    # Return the processed image as a PyTorch tensor
    return_tensors="pt"

    # Extract only the processed image tensor (pixel_values) from the returned dictionary.
)["pixel_values"]

# Display the shape of the processed image tensor.
processed_image.shape
```

```
torch.Size([1, 3, 224, 224])
```

```
# Visualizing the processed image
```

```
# Import PyTorch library for tensor operations.
import torch

# Import NumPy for array operations.
import numpy as np

# Import Matplotlib to display the image.
import matplotlib.pyplot as plt

# Prepare image for visualization
# Remove the batch dimension from the processed image.
# Shape changes from [1, 3, 224, 224] → [3, 224, 224].
img = processed_image.squeeze(0)

# Converts the image from [Channels, Height, Width] to [Width, Height, Channels].
img = img.permute(*torch.arange(img.ndim - 1, -1, -1))

# Swap the first two dimensions.
# Converts [Width, Height, Channels] → [Height, Width, Channels]
img = np.einsum("ijk->jik", img)

# Visualize preprocessed image
plt.imshow(img)

# Hide the x-axis and y-axis values.
plt.axis("off")
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for (np.float64(-0.5), np.float64(223.5), np.float64(223.5), np.float64(-0.5))



```
# Generate a 512-dimensional image embedding from the preprocessed image using the CLIP image encoder
image_embedding = model.get_image_features(processed_image)
processed_image.shape
```

```
torch.Size([1, 3, 224, 224])
```

```
# Extract the pooled (final) text embedding representing the entire input sentence.
# When using CLIPTextModel or CLIPVisionModel, the model returns multiple outputs,
# so we use .pooler_output to extract the single embedding that represents the entire text or image.
text_embedding = text_embedding.pooler_output

image_embedding = image_embedding.pooler_output
```

```
# Normalize the text embedding by dividing each embedding value by the vector's magnitude (L2 norm)
text_embedding /= text_embedding.norm(dim=-1, keepdim=True)

# Normalize the image embedding by dividing each embedding value by the vector's magnitude (L2 norm).
image_embedding /= image_embedding.norm(dim=-1, keepdim=True)

# Stop gradient tracking, move the text embedding to the CPU, and convert it to a NumPy array.
text_embedding = text_embedding.detach().cpu().numpy()

# Stop gradient tracking, move the image embedding to the CPU, and convert it to a NumPy array.
image_embedding = image_embedding.detach().cpu().numpy()

# Calculate the similarity score by taking the dot product of the normalized text and image embeddings.
score = np.dot(text_embedding, image_embedding.T)
```

```
score
```

```
array([[0.31460106]], dtype=float32)
```

```
from PIL import Image

# Load the images and convert it to RGB format
puppy = Image.open("/content/puppy.jpg").convert("RGB")
cat = Image.open("/content/cat.jpg").convert("RGB")
car = Image.open("/content/car.jpg").convert("RGB")

# Store all the images in a list.
images=[puppy,cat,car]

# Store the corresponding captions for each image.
captions=["a puppy playing in the snow"
"a cute cat"
"car on the road with sunset background"]
```

```
#          ===== USING SENTENCE-TRANSFORMERS TO LOAD CLIP =====
```

```
# Import the SentenceTransformer class to load pretrained embedding models,
# and util to compute cosine similarity between embeddings.
from sentence_transformers import SentenceTransformer, util

# Load the CLIP model that is compatible with the Sentence Transformers library.
model = SentenceTransformer("clip-ViT-B-32")

# Generate embeddings for all the input images.
image_embeddings = model.encode(images)

# Generate embeddings for all the input text captions.
text_embeddings = model.encode(captions)

# Compute the cosine similarity between every image embedding
# and every text embedding, resulting in a similarity matrix.
sim_matrix = util.cos_sim(
    image_embeddings,
    text_embeddings
)

# Display the similarity matrix.
print(sim_matrix)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as the secret `HF_TOKEN` in your Colab secrets, and restart this notebook.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
```

```
modules.json: 100% 122/122 [00:00<00:00, 12.9kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 11.0kB/s]

README.md: 1.91k/? [00:00<00:00, 178kB/s]

config.json: 4.03k/? [00:00<00:00, 59.8kB/s]

merges.txt: 525k/? [00:00<00:00, 5.84MB/s]

preprocessor_config.json: 100% 316/316 [00:00<00:00, 4.70kB/s]

special_tokens_map.json: 100% 389/389 [00:00<00:00, 5.67kB/s]

vocab.json: 961k/? [00:00<00:00, 13.7MB/s]

tokenizer_config.json: 100% 604/604 [00:00<00:00, 7.80kB/s]

0_CLIPModel/model.safetensors: 100% 605M/605M [00:18<00:00, 76.1MB/s]

0_CLIPModel/pytorch_model.bin: 100% 605M/605M [00:17<00:00, 29.9MB/s]

tensor([[0.2696],
        [0.2054],
        [0.2630]])
```

```
#          Making Text Generation Models Multimodal
#          Preprocessing Multimodal Inputs
```

```
# Import AutoProcessor to preprocess both images and text,
# and Blip2ForConditionalGeneration to load the pretrained BLIP-2 model.
from transformers import AutoProcessor, Blip2ForConditionalGeneration
```

```
# Import PyTorch for tensor operations and GPU support.
import torch

# Load the BLIP-2 processor, which preprocesses images and text
# into the format expected by the model.
blip_processor = AutoProcessor.from_pretrained(
    "Salesforce/blip2-opt-2.7b"
)

# Load the pretrained BLIP-2 model for image captioning
# and visual question answering using 16-bit floating-point precision.
model = Blip2ForConditionalGeneration.from_pretrained(
    "Salesforce/blip2-opt-2.7b",
    torch_dtype=torch.float16
)

# Select GPU if available, otherwise CPU
device = "cuda" if torch.cuda.is_available() else "cpu"

# Move model to selected device
model.to(device)
```



```

processor_config.json: 100% 68.0/68.0 [00:00<00:00, 5.76kB/s]
preprocessor_config.json: 100% 432/432 [00:00<00:00, 48.7kB/s]
tokenizer_config.json: 100% 882/882 [00:00<00:00, 88.1kB/s]
vocab.json: 798k/? [00:00<00:00, 26.9MB/s]
merges.txt: 456k/? [00:00<00:00, 22.4MB/s]
tokenizer.json: 3.56M/? [00:00<00:00, 84.5MB/s]
added_tokens.json: 100% 23.0/23.0 [00:00<00:00, 2.12kB/s]
special_tokens_map.json: 100% 548/548 [00:00<00:00, 48.4kB/s]
config.json: 1.03k/? [00:00<00:00, 92.2kB/s]
model.safetensors.index.json: 122k/? [00:00<00:00, 10.7MB/s]
Downloading shards: 100% 2/2 [08:16<00:00, 233.57s/it]
model-00001-of-00002.safetensors: 100% 10.0G/10.0G [05:30<00:00, 23.2MB/s]
model-00002-of-00002.safetensors: 100% 4.98G/4.98G [02:44<00:00, 31.7MB/s]
Loading checkpoint shards: 100% 2/2 [01:02<00:00, 29.56s/it]
generation_config.json: 100% 141/141 [00:00<00:00, 11.0kB/s]
Blip2ForConditionalGeneration(
  (vision_model): Blip2VisionModel(
    (embeddings): Blip2VisionEmbeddings(
      (patch_embedding): Conv2d(3, 1408, kernel_size=(14, 14), stride=(14, 14))
    )
    (encoder): Blip2Encoder(
      (layers): ModuleList(
        (0-38): 39 x Blip2EncoderLayer(
          (self_attn): Blip2Attention(
            (dropout): Dropout(p=0.0, inplace=False)
            (qkv): Linear(in_features=1408, out_features=4224, bias=True)
            (projection): Linear(in_features=1408, out_features=1408, bias=True)
          )
          (layer_norm1): LayerNorm((1408,), eps=1e-06, elementwise_affine=True)
          (mlp): Blip2MLP(
            (activation_fn): GELUActivation()
            (fc1): Linear(in_features=1408, out_features=6144, bias=True)
            (fc2): Linear(in_features=6144, out_features=1408, bias=True)
          )
          (layer_norm2): LayerNorm((1408,), eps=1e-06, elementwise_affine=True)
        )
      )
    )
    (post_layernorm): LayerNorm((1408,), eps=1e-06, elementwise_affine=True)
  )
  (qformer): Blip2QFormerModel(
    (layernorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
    (encoder): Blip2QFormerEncoder(
      (layer): ModuleList(
        (0): Blip2QFormerLayer(
          (attention): Blip2QFormerAttention(
            (attention): Blip2QFormerMultiHeadAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): Blip2QFormerSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
      )
    )
    (crossattention): Blip2QFormerAttention(
      (attention): Blip2QFormerMultiHeadAttention(
        (query): Linear(in_features=768, out_features=768, bias=True)
        (key): Linear(in_features=1408, out_features=768, bias=True)
        (value): Linear(in_features=1408, out_features=768, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (output): Blip2QFormerSelfOutput(
        (dense): Linear(in_features=768, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
    (intermediate_query): Blip2QFormerIntermediate(
      (dense): Linear(in_features=768, out_features=3072, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output_query): Blip2QFormerOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)

```

```

        (dense): Linear(in_features=3072, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
    )
)
(1): Blip2QFormerLayer(
  (attention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=768, out_features=768, bias=True)
      (value): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate_query): Blip2QFormerIntermediate(
    (dense): Linear(in_features=768, out_features=3072, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output_query): Blip2QFormerOutput(
    (dense): Linear(in_features=3072, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
(2): Blip2QFormerLayer(
  (attention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=768, out_features=768, bias=True)
      (value): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (crossattention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=1408, out_features=768, bias=True)
      (value): Linear(in_features=1408, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate_query): Blip2QFormerIntermediate(
    (dense): Linear(in_features=768, out_features=3072, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output_query): Blip2QFormerOutput(
    (dense): Linear(in_features=3072, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
(3): Blip2QFormerLayer(
  (attention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=768, out_features=768, bias=True)
      (value): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate_query): Blip2QFormerIntermediate(
    (dense): Linear(in_features=768, out_features=3072, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output_query): Blip2QFormerOutput(
    (dense): Linear(in_features=3072, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
(4): Blip2QFormerLayer(

```

```

(attention): Blip2QFormerAttention(
  (attention): Blip2QFormerMultiHeadAttention(
    (query): Linear(in_features=768, out_features=768, bias=True)
    (key): Linear(in_features=768, out_features=768, bias=True)
    (value): Linear(in_features=768, out_features=768, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (output): Blip2QFormerSelfOutput(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
(crossattention): Blip2QFormerAttention(
  (attention): Blip2QFormerMultiHeadAttention(
    (query): Linear(in_features=768, out_features=768, bias=True)
    (key): Linear(in_features=1408, out_features=768, bias=True)
    (value): Linear(in_features=1408, out_features=768, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (output): Blip2QFormerSelfOutput(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
(intermediate_query): Blip2QFormerIntermediate(
  (dense): Linear(in_features=768, out_features=3072, bias=True)
  (intermediate_act_fn): GELUActivation()
)
(output_query): Blip2QFormerOutput(
  (dense): Linear(in_features=3072, out_features=768, bias=True)
  (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
  (dropout): Dropout(p=0.1, inplace=False)
)
)
(5): Blip2QFormerLayer(
  (attention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=768, out_features=768, bias=True)
      (value): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate_query): Blip2QFormerIntermediate(
    (dense): Linear(in_features=768, out_features=3072, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output_query): Blip2QFormerOutput(
    (dense): Linear(in_features=3072, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
)
(6): Blip2QFormerLayer(
  (attention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=768, out_features=768, bias=True)
      (value): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (crossattention): Blip2QFormerAttention(
    (attention): Blip2QFormerMultiHeadAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=1408, out_features=768, bias=True)
      (value): Linear(in_features=1408, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): Blip2QFormerSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate_query): Blip2QFormerIntermediate(
    (dense): Linear(in_features=768, out_features=3072, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
)

```

```

    )
    (output_query): Blip2QFormerOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (7): Blip2QFormerLayer(
    (attention): Blip2QFormerAttention(
      (attention): Blip2QFormerMultiHeadAttention(
        (query): Linear(in_features=768, out_features=768, bias=True)
        (key): Linear(in_features=768, out_features=768, bias=True)
        (value): Linear(in_features=768, out_features=768, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (output): Blip2QFormerSelfOutput(
        (dense): Linear(in_features=768, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
    (intermediate_query): Blip2QFormerIntermediate(
      (dense): Linear(in_features=768, out_features=3072, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output_query): Blip2QFormerOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (8): Blip2QFormerLayer(
    (attention): Blip2QFormerAttention(
      (attention): Blip2QFormerMultiHeadAttention(
        (query): Linear(in_features=768, out_features=768, bias=True)
        (key): Linear(in_features=768, out_features=768, bias=True)
        (value): Linear(in_features=768, out_features=768, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (output): Blip2QFormerSelfOutput(
        (dense): Linear(in_features=768, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
    (crossattention): Blip2QFormerAttention(
      (attention): Blip2QFormerMultiHeadAttention(
        (query): Linear(in_features=768, out_features=768, bias=True)
        (key): Linear(in_features=1408, out_features=768, bias=True)
        (value): Linear(in_features=1408, out_features=768, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (output): Blip2QFormerSelfOutput(
        (dense): Linear(in_features=768, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
    (intermediate_query): Blip2QFormerIntermediate(
      (dense): Linear(in_features=768, out_features=3072, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output_query): Blip2QFormerOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (9): Blip2QFormerLayer(
    (attention): Blip2QFormerAttention(
      (attention): Blip2QFormerMultiHeadAttention(
        (query): Linear(in_features=768, out_features=768, bias=True)
        (key): Linear(in_features=768, out_features=768, bias=True)
        (value): Linear(in_features=768, out_features=768, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (output): Blip2QFormerSelfOutput(
        (dense): Linear(in_features=768, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
    (intermediate_query): Blip2QFormerIntermediate(
      (dense): Linear(in_features=768, out_features=3072, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output_query): Blip2QFormerOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )

```